

IIT JODHPUR

Feb'26 Minor Examination: EEL71090 ML in VLSI CAD

Guidelines (Total time: 02 Hours, Maximum Marks: 20):

- Please read the question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume whatever you wish to and state that in your answer.
 - Usage of LLM/AI tool is NOT allowed. NOTES/BOOKS can be referred to. For Offline students, access to LAPTOP/MOBILE is NOT allowed.
-

1. A semiconductor company is developing a parameterized RTL implementation of a multi-channel DMA controller integrated with a RISC-V ISA-based SoC. The DMA supports burst transfers (i.e., multi-byte transfers), priority arbitration, and configurable FIFO depths. Traditional constrained-random verification using SV/UVM is failing to efficiently reach corner-case coverage related to:
 - FIFO overflow/underflow
 - Arbitration starvation
 - Back-to-back burst boundary conditions

The verification team observes that:

- Only 72% functional coverage is achieved after 48 hours of simulation.
- Certain cross-coverage bins (burst length $\times$ priority $\times$ FIFO depth) remain unhit.
- Directed test writing is becoming infeasible due to state-space explosion.

Propose a machine learning-based framework to guide test generation for coverage closure. Your answer should clearly describe the following:

- What data need to be collected from simulations? [2]
 - How features will be extracted from RTL signals and coverage reports? [2]
 - Whether you would use supervised learning, reinforcement learning, or unsupervised learning — and why? Explain with an example in the above setting. [2]
2. A deep-tech startup team is using Siemens EDA Catapult HLS to synthesize C/C++ descriptions of compute kernels (e.g., matrix multiplication, FIR filter, CNN layers) into RTL targeting a 7nm ASIC standard-cell library.

The team observes that different combinations of HLS directives:

- Loop unrolling factors
- Loop pipelining (II constraints)
- Array partitioning strategies
- Resource binding (DSP vs LUT)

Each method produces significantly different Quality of Results (QoR) in terms of:

- Latency (cycles)
- Area (gate count / DSP count) and Power
- Achievable clock frequency

However, running full HLS synthesis for each directive configuration takes 20–45 minutes, making design space exploration (DSE) extremely slow.

You are tasked with designing a Graph Neural Network (GNN)-based framework to predict QoR before running the full synthesis.

Explain how the C/C++ HLS input can be converted into a graph suitable for GNN processing.

- What will the nodes and edges represent? (e.g., operations, loops, memory accesses, control blocks, data dependency, control dependency, memory dependency etc.) [1.5]
- How would you encode HLS directives (e.g., unroll factor = 3 or 4) into the graph? [1.5]
- Describe how would you extract the node-level and graph-level features? [2]
- Design a GNN architecture that predicts multiple QoR metrics simultaneously. [2]
- Would you use Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), or Message Passing Neural Networks (MPNN)? [1.5]

3. A startup *MewarTech* incubated at IITJ is performing logic synthesis of a complex SoC subsystem written in SystemVerilog (e.g., a multi-core cache-coherent interconnect) using Synopsys Design Compiler targeting a 5nm standard-cell library. The synthesis process converts RTL into a gate-level netlist using following steps:

- Technology mapping
- Boolean restructuring
- Logic balancing
- Retiming
- Gate sizing
- Buffer insertion

The final Quality of Results (QoR) is evaluated in terms of:

- Power (dynamic + leakage) and Area (total cell area)
- Performance (critical path delay / Fmax)

However, the design team in this startup observes that:

- Significant QoR variability depending on synthesis effort level and constraint tuning
- Manual tuning of synthesis scripts is time-consuming
- PPA trade-offs are difficult to predict before full synthesis of the design

You are asked to design a Deep Learning (DL)-based framework that predicts and optimizes PPA during RTL-to-gate logic synthesis

(a) Explain with inputs/outputs (of respective model) if this is a regression problem, ranking problem, or reinforcement learning problem? [1.5]

(b) Propose how RTL or intermediate representations (e.g., And-Inverter Graphs (AIG), Boolean networks, or technology-mapped netlists) can be converted into a graph for deep learning. [2]

(c) Design a framework where DL assists synthesis in predicting beneficial transformations (e.g., retime vs. restructure etc.)? [2]